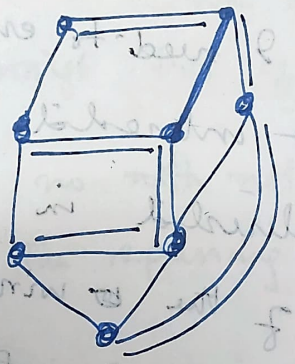
Greedy algorithm

Minimum spanning trees.

Given an undirected graph $G(V, E)$, spanning tree in G which is subgraph that is a tree and which includes all the vertices of G.

Spanning → should include all the vertices.

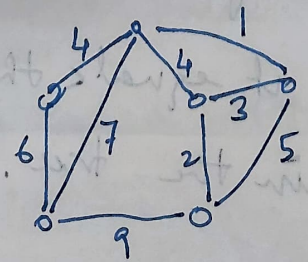
tree - Connected subgraph without cycle.



spanning tree has $|V|-1$ edges.

Now what is Minimum spanning tree.: Minimum spanning tree of a graph is a spanning tree of the graph of minimum weight / length / cost not our given the graph $G(V, E)$ but are a length function l.

such that $l : E \to \mathbb{R}^+$. we assume to non negative weight



Think the problem this way, there are certain cities and we need to connect them by wire

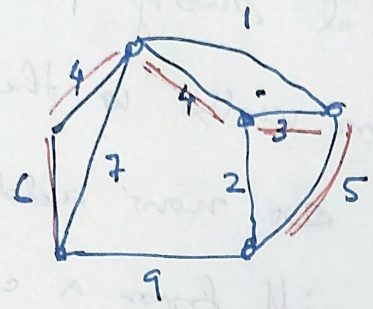
So we have plenty of options to connect the cities by wire. Some of the options are not possible due to some unfortunate reason.

So this the graph so connect 2 cities by wire I am also told how much ~~wire~~ length of the wire do I need to connect them.

Suppose I @ the length of the wire needed are as shown in the figure. So we need to connect the cities which means I need to create spanning trees and I am not interested in any spanning tree. I am interested in spanning tree for which the length of the wire used is as small as possible. So that is what we mean by minimum spanning tree, a spanning tree of min length. So what is the length of the ~~tree~~ spanning tree now, it is nothing but the sum of the lengths of the edges in the tree.

Let us formally define that the length of the spanning tree: it equals the sum of the lengths of edges in the tree.

So I might now decide to pick some edges suppose I pick.



Let us pick the edges as shown.

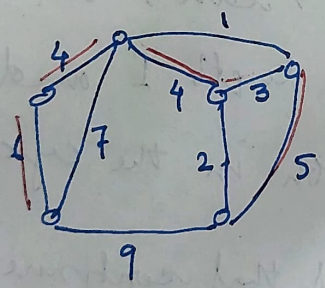
what is the length of the tree?

$6+4+4+3+5 = 22$

there might be other spanning tree which are smaller than 22.

We are interested in finding the min spanning tree of smallest weight.

There is no notion of min root in the case of the spanning tree.

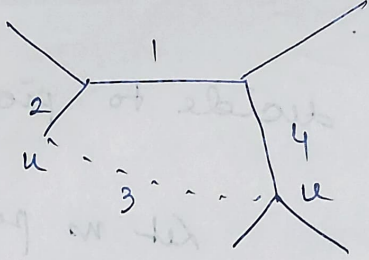


one way you convince yourself that this is not the no smallest possible tree is by seeing that for instance.

If I include the edge of length 1 then what is going to happen:

will it still be a tree?

No we will have a cycle. So that is one property of a spanning tree always.



Suppose you have this spanning tree.

Spanning tree means connected subgraph that between every two vertex there is a path already. If we now add one more edge now, it will form a cycle, Because between u and v there is already a path. So addy the edge between u and v (shown dotted in) it will form a cycle ie mean it no longer remain a spanning tree.

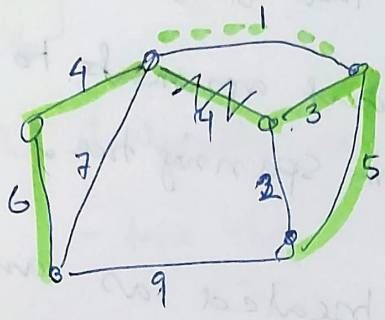
Now suppose in this cycle each has certain length. suppose the weights are as shown. Now what can I do? I can include the edge with length 1 and drop the edge of max length in the cycle.

Now question is will that continue to be a tree.

Why would it remain connected?

I can try to add an edge into the tree and see as result of that addition I will clearly from a cycle and if from the cycle I can drop and edge of longer length than cost of the tree.

For instance if 9 and the edges of length 1

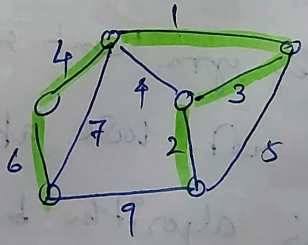


Then 9 would form a cycle. Now I can drop any edge of the cycle and it still remains connected. Now which edge I would like to drop. ? edge with cost 4

Now the cost of the tree becomes 22-3 = 19

So I can repeat the process and see if there ~~anything~~ else that 9 can drop.

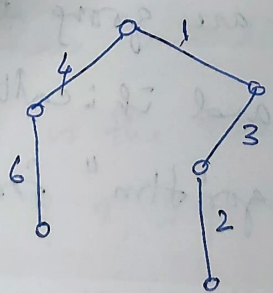
Now we can drop 5 and include 2



Now the weight becomes

19 - 3 = 16

So the tree that 9 have at this point is



would it more sense to include other edge and try and repeat the process?

No : explanation

So it does not more sense to include other edge.

and so this is the minimum, I should not say so this is not the reason that it is min spanning tree. There are other different arguments to prove that this is the minimum spanning tree.

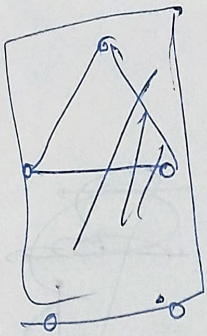
So this could be treated as an algorithm to compute the minimum spanning tree.

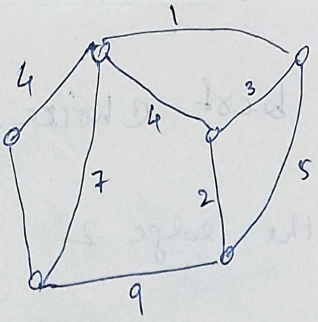
How would it work?

You start with a tree, then try to add an edge look at the cycle gets formed. See if you can drop and edge of longer length from that cycle. Therefor reduce the cost, so you can keep doing this procedure till you reduce the cost and stop when you cant reduce any more. But we will not look at this algorithm, or analys this algorithm because it will be fairly expensive in terms of running time.

So we are going to look at different algorithm. and it is called "Kruskal algorithm" for computing the MST

Let me illustrate on the same graph that we had before.





So the ~~A~~ algorithm says the following

- take the minimum edge of the graph

The algorithm is called greedy.

So what do you think greedy means here. ~~+~~ Make the best possible choice without thinking of the future. Thats the key thing

Here our objective is to minimize the length of the tree so what is the best thing to do at the 1st step.

- ~~Take~~ Pick the edge with smallest length. and it is included in the min spanning tree.

we are constructing the tree edge by edge

1

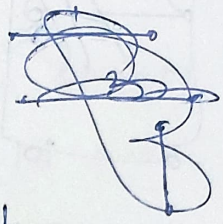
So we have included the edge of length 1

Now which is the next ~~smallest~~ best choice to make? { 2, 3 or what }

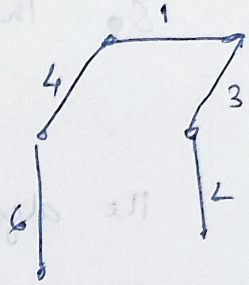
If you think that it should remain connected then you are looking at the future. dont do that. be greedy, as you can.

just make the best choice.

Next best is the edge 2



No what does greedy say, which is the next best choice? edge 3 right, so it is also included in the MST



Next 4,

There are two, pick one

Now if we pick the edge with 4 it will form a cycle. so if we include this it will not be there any more. So we will not include this edge.

So if inclusion of an edge from a cycle I dont include that edge.

Next I try to include 5 but that also form a cycle so we wont include that edge.

what next? 6

Now we can stop at this point as now we have covered all the vertex and it is the spanning tree. any other edge that I try to include from a cycle.

because this is a spanning tree. We have not yet argued that this is the best possible tree. Till now we have not seen an argument for why this is the best possible but this is the same solution that we obtained earlier.

Let us quickly write down what the algorithm is.

Kruskal algorithm for MST

1. Sort edges in increasing order of length

   Let $e_1, e_2, e_3, \ldots e_m$

   $l(e_i) \leq l(e_{i+1})$

   $T \leftarrow \phi$ (set of edges)

2) $i = 1$ to m do

   if $e_i \cup T$ is a tree

   $T \leftarrow T \cup e_i$

3. Return T

How do you check that the edge that you are trying to include forms a cycle or not?

This is very important step and it will dictate the running time of the algorithm.

Q. Solving this m log n

If the check of cycle takes m log n time then the algorithm would be a m log n algorithm.

Before that we have to argue why this is the MST?

Why there could not be any other spanning tree smaller than this?

Proof

We will look at the edges picked by Kruskal algorithm

Let us name the edges picked by Kruskal algorithm as

$$g_1 < g_2 < g_3 \ldots < g_{n-1}$$

Suppose some how picked the best spanning tree by searching the graph in brute force way and let the name of the edges of that MST (optimum tree) are

$$f_1 < f_2 < f_3 \ldots < f_{n-1}$$

assume all edge lengths are distinct for simplicity

Now these two sets of edges need not be the same. What we will argue is that

they are indeed the same. If they are one and the same then what we have found out by the Kruskal algorithm is the best tree.

Proof is by contradiction

Suppose these sets differ & the 1st place where they differ is i

$$\begin{array}{lllll} g_1 < g_2 < g_3 \cdots & g_{i-1} & < g_i & \cdots & g_{m-1} \\ \| \quad \| \quad \| & \| & & & \\ f_1 < f_2 < f_3 \cdots & f_{i-1} & < f_i & \cdots & f_{m-1} \end{array}$$

We start from the left and see which edge differs

$g_1 = f_1 ; g_2 = f_2 \cdots g_{i-1} = f_{i-1}$ but

$$g_i \neq f_i$$

We will argue that this can not happen

There are two possibilities now

i) $g_i < f_i$

ii) $g_i > f_i$

Lemma 1

$l(g_i) < l(f_i)$

$g_i = f_i$ can not happen because all edges are of distinct length

all the edge after f_i are longer than f_i

So g_i can not be there in $f_{i+1} \dots f_{m-1}$

and it can not be $f_1 \dots f_{i-1}$ either.

because $f_i \dots f_{i-1}$ are same as $g_1 \dots g_{i-1}$ respectively as g_i is and they are all distinct

that mean g_i is not then in $f_1 \dots f_{m-1}$

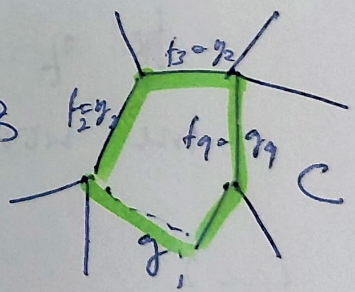
ie g_i is not in the optimum tree.

So we can add g_i in the optimum tree.

add g_i to opt tree

it could form a cycle

let the cycle be C.



can it happen that g_i is of longest length in the cycle C?

if g_i is it can happen if all the edges in C are in $f_1 \dots f_{i-1}$

then would have formed cycle in the Algorithm founded tree. because f_1 to f_{i-1} are identical to g_1 to g_{i-1}

g_i can only be larger in the cycle C if all other edges in the cycle are from f_1 to $\cdots f_{i-1}$.

But $f_1 \cdots f_{i-1}$ are identical to $g_1 \cdots g_{i-1}$

So if g_i has formed a cycle in Optree then it would have formed cycle in the tree found by Kruskal algorithm

If any edge in the cycle C is larger than g_i then we are done.

Why?

we will include g_i and remove the edge of larger length and cost of the optimum tree would reduce, and get a contradiction

Case 2

$g_i > f_i$

f_i is distinct from $f_1 \cdots f_{i-1}$

also means f_i is distinct from $g_1 \cdots g_{i-1}$

Now question is why didn't we pick g_i in the Kruskal algorithm?

It could have formed a cycle

ie

$f_i \cup \{g_1 \dots g_{i-1}\}$ contains a cycle

$= f_i \cup \{f_1 \dots f_{i-1}\}$ also contains a cycle

$\Rightarrow$ opt also contains a cycle

but that is not possible, ~~so~~ it's a tree

So thats the contradiction

So we can say that ~~thay~~ there is no place where these two place differ.

There two edges are one and the same

$\therefore$ Kruskal algorithm produce MST.

Now

is the MST unique?

Yes if edge lengths are distinct

No otherwise.